Rapport d'optimisation — TP R3.14

Étudiant : Soane ILOAI — BUT MMI Béziers

Site déployé : https://soane986.github.io/R314-optimisation-site/

Fork (historique des commits) : https://github.com/Soane986/R314-optimisation-site

Site de référence (avant) : https://nico3807.github.io/R314-optimisation-site/

> Mesures réalisées le 25 octobre 2026 entre 02 h 47 et 03 h 20. Les captures d'écran correspondantes sont rangées dans le dossier `audit/`.

---

1. Conditions de mesure

Paramètre	Valeur
Outil principal	**PageSpeed Insights** (pagespeed.web.dev), Lighthouse 13.5.0, Chromium headless 153
Profil mobile	Émulation Moto G Power, connexion 4G lente, processeur ralenti
Profil ordinateur	Profil « Ordinateur » de PageSpeed Insights
Répétitions	**3 analyses par site et par profil**, valeur **médiane** retenue
Avant	Site de référence `nico3807.github.io` (non modifié)
Après	Mon fork publié `soane986.github.io`. Pour les analyses 2 et 3, j'ai ajouté `?run=2` et `?run=3` à l'adresse, car PageSpeed renvoyait sinon le résultat en cache de l'analyse précédente. La page est strictement la même.
Mesures complémentaires	Navigateur Chromium, fenêtre de **1280 × 800 px**, sans ralentissement processeur. Elles servent à mesurer ce que Lighthouse ne voit pas : les blocages après 6,5 s et le coût du défilement.

Limite méthodologique : PageSpeed Insights n'affiche aucune donnée de terrain (rapport CrUX), car le site n'a pas assez de trafic réel. L'audit repose donc entièrement sur des données de laboratoire, c'est-à-dire des conditions simulées.

Poids en local et en ligne : les fichiers texte (HTML, CSS, JS) sont servis compressés en gzip par GitHub Pages, mais pas par un serveur local. Tous les poids de la section 2 ont donc été relevés sur les sites **en ligne**.

---

2. Tableau comparatif des six indicateurs

Médiane de 3 analyses PageSpeed Insights. Le détail des 3 essais est donné plus bas.

Indicateur	Avant — Mobile	Après — Mobile	Avant — Desktop	Après — Desktop
Poids total transféré	16 376 Kio	**559 Kio** (−96,6 %)	16 375 Kio	**210 Kio** (−98,7 %)
Requêtes (au chargement)	19, dont 1 en 404	**9**	19, dont 1 en 404	**9**
FCP	2,4 s	**2,4 s** (inchangé)	0,9 s	**0,6 s**
LCP	67,1 s	**3,0 s**	10,9 s	**0,6 s**
CLS	0	**0**	0	**0**
TBT	100 ms	**0 ms**	320 ms	**0 ms**
Speed Index	17,9 s	**4,4 s**	3,9 s	**0,6 s**
Score Performance	**61**	**89**	**52**	**100**

Détail des 3 essais

Profil	Site	Score	FCP	LCP	TBT	CLS	Speed Index
Mobile	Avant	61 · 62 · 60	2,4 · 2,4 · 2,4 s	67,1 · 67,0 · 67,1 s	90 · 100 · 180 ms	0 · 0 · 0	19,8 · 15,8 · 17,9 s
Mobile	Après	89 · 94 · 87	2,4 · 2,4 · 2,4 s	3,0 · 2,6 · 3,2 s	0 · 0 · 0 ms	0 · 0,019 · 0	4,4 · 2,4 · 4,5 s
Desktop	Avant	50 · 58 · 52	0,9 · 0,9 · 1,0 s	12,2 · 10,6 · 10,9 s	340 · 210 · 320 ms	0,021 · 0 · 0	3,9 · 4,0 · 3,3 s
Desktop	Après	100 · 100 · 99	0,6 · 0,6 · 0,7 s	0,6 · 0,6 · 0,7 s	0 · 0 · 0 ms	0 · 0 · 0	0,6 · 0,6 · 0,7 s

Requêtes : pour les deux sites, je les ai comptées dans le navigateur (fichiers chargés par la page, plus le document). Avant : `polyfills.js` en 404, `img6.jpg` téléchargée deux fois, 3 feuilles CSS en chaîne et 2 feuilles Google Fonts. Après : les 3 images de la 2e rangée ne sont chargées qu'à l'approche de l'écran (`lazy`).

Mesures complémentaires (navigateur, fenêtre 1280 × 800 px)

Mesure	Avant	Après
Blocages du fil principal au chargement (> 50 ms)	691 ms et 454 ms	**aucun**
Blocages entre 6 et 30 s (`analytics.js`)	66 → 87 → 115 → 149 → 194 → 251 ms, toutes les 2,6 s, de plus en plus longs	**aucun** (le suivi tourne toujours : 9 passages enregistrés)
Temps total de blocage sur 30 s	**2 007 ms**	**0 ms**
JavaScript exécuté par événement de défilement	**19,2 ms** (au-dessus du budget de 16,7 ms par image)	**0,03 ms** (le calcul est reporté à une fois par image : 0,12 ms)

Méthode : pour les blocages, une minuterie de 10 ms tourne pendant 30 s, et tout retard de plus de 50 ms est un blocage du fil principal. Pour le défilement, 30 événements `scroll` sont déclenchés et chronométrés.

---

3. Problèmes identifiés et corrections

Les numéros de ligne renvoient au code **d'origine**. Chaque correction correspond à un commit du fork.

Niveau 1 — visibles dans l'onglet Réseau et Lighthouse

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
1	`index.html:75-90`, `img1-6.html`	Photos en 2040 à 3968 px affichées dans des cartes d'environ 388 px ; aucun `width` / `height`, aucun `lazy`	PSI : « Améliorer l'affichage des images, économies estimées 10 769 Kio » ; Réseau trié par taille	9,9 Mo d'images	Variantes de 400, 800 et 1200 px (JPEG q78), `srcset` / `sizes` calculés à partir des media queries, `width` / `height`, `loading="lazy"` sur la 2e rangée, `fetchpriority="high"` sur la 1re vignette — *Redimensionne et compresse les photos*
2	`img1-6.html`	Image des pages de détail en taille native, qui fait défiler la page horizontalement	Mode appareil mobile (Ctrl+Maj+M)	Lisibilité, et jusqu'à 3,4 Mo	Règle `.image-wrapper img{max-width:100%;height:auto}` — même commit
3	`index.html:12`	`preload` de `img6.jpg?v=2` : URL différente de `img6.jpg`, donc **téléchargée deux fois** ; de plus, c'est une image hors de l'écran	Deux lignes `img6` dans la liste des requêtes	3,39 Mo, et de la bande passante prise aux ressources critiques	Préchargement supprimé — même commit

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
4	`css/theme-legacy.css:11`	Texture de 3968 px (3,4 Mo, même fichier que img6) affichée à **4 % d'opacité**	Image présente dans les requêtes mais introuvable à l'écran	3,39 Mo	Version 640 px de 37 Ko, rendu identique à cette opacité (effet conservé) — *Allège favicon et texture hero*
5	`index.html:8`	Favicon PNG de 512 px (787 Ko)	Onglet Réseau	787 Ko	PNG de 32 px (2 Ko) — même commit
6	`index.html:16`	`js/legacy/polyfills.js` en **404**, script **bloquant** dans le `<head>`	Requête en statut 404	Un aller-retour bloquant pour rien	Référence supprimée — *Supprime préchargements et requêtes inutiles*
7	`index.html:9`	`dns-prefetch` vers `cdn.jsdelivr.net`, jamais contacté	Ce domaine n'apparaît dans aucune requête	Une résolution DNS inutile	Supprimé — même commit
8	`index.html:10-11`	`preload` de police **sans** `crossorigin`, qui pointe sur la graisse 300 (`pxiByp8kv8JHgFVrLDz8Z1xlFQ`), inutilisée	Comparaison avec la feuille Google Fonts : la 400 est `pxiEyp8…`, la 600 est `pxiByp8… LEj6…`	Police téléchargée pour rien	Supprimé — même commit
9	`css/styles.css:1-2`, `extra.css:1`, `theme-legacy.css:3`	Chaîne de 4 `@import` en série (dont Google Fonts deux fois)	PSI : « Requêtes de blocage du rendu, économies estimées 1 820 ms »	Allers-retours en série avant le premier rendu	Un seul fichier CSS ; Poppins chargée par `<link>` avec `preconnect` — *Fusionne CSS, polices non bloquantes*
10	`css/styles.css:2`	Police en `display=block` (texte invisible au chargement) et graisse 300 inutilisée	Lighthouse : « affichage des polices »	Texte invisible pendant le chargement	`display=swap`, graisses 400 et 600 seulement (le gras 700 reste rendu en 600, comme avant) — même commit
11	`css/styles.css` (1199 règles), `extra.css` (800 règles), `theme-legacy.css`	Règles `.redundant-*`, `.extra-*` (`filter`), `.tile`, `.legacy-*` jamais utilisées	Aucune occurrence dans les fichiers HTML et JS	117 Ko de CSS bloquant	Supprimées : 122 Ko → 5 Ko — *Supprime règles CSS jamais utilisées*

Niveau 2 — visibles seulement dans un profil d'exécution

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
12	`js/vendor/ui-kit.min.js` (code déplié)	Au chargement : boucle `while` de **620 ms** sur `performance.now()` et création de **240 000 objets** jamais utilisés	Blocages de 691 et 454 ms au chargement (mesure navigateur) ; PSI : « 2 tâches longues trouvées »	TBT de 320 ms en desktop	Librairie supprimée : `app.js` n'utilisait que `qs`, `qsa`, `cls` et `ready`, remplacés par l'API native — *Remplace UIKit par JavaScript natif*

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
13	`js/vendor/ui-kit.min.js`	`MutationObserver` sur tout le document (`subtree` , `attributes`) qui appelle `getComputedStyle()` sur **tous** les éléments à chaque mutation	Lecture du code déplié (bouton `{}` du panneau Sources) ; contribue aux 19,2 ms par événement scroll	Multiplié par le n°14 : chaque style écrit au scroll relance l'observateur	Supprimé avec la librairie — même commit
14	`js/app.js:157-188` (`revealCards`)	Appelée à **chaque** événement scroll et resize : `querySelectorAll` et lecture de `offsetHeight` de toutes les cartes **à l'intérieur** de la boucle sur les cartes ($O(n^2)$), lectures et écritures de style alternées (**reflow forcé**), puis `getBoundingClientRect` sur les **371** cases du calendrier	PSI : « Ajustement forcé de la mise en page » ; **19,2 ms par événement scroll**	Défilement saccadé	`IntersectionObserver` pour l'apparition des cartes ; calendrier : 1 lecture de géométrie et 7 variables CSS (une par ligne), écouteur `passive` limité à une fois par image — *Supprime reflows forcés au scroll*. Résultat : **0,03 ms par événement**
15	`js/app.js:81` (`buildKeywordIndex`)	`innerHTML +=` dans une boucle de 320 mots-clés : toute la liste est re-analysée à chaque tour (coût quadratique)	Lecture du code : `innerHTML +=` dans la boucle (cas C5 du guide)	51 360 options créées au lieu de 320	`DocumentFragment` et une seule insertion — *Construit suggestions avec fragment unique*
16	`index.html:17-18` , `img1-6.html`	`ui-kit.min.js` et `app.js` chargés sans `defer` dans le `<head>` ; sur les pages de détail, ils ne servent à rien	PSI : « Requêtes de blocage du rendu »	Analyse du HTML bloquée	`app.js` en `defer` ; scripts retirés des pages de détail — *Remplace UIKit par JavaScript natif*

Niveau 3 — conditionnel

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
17	`js/analytics.js:19-29` (code obscurci)	Les chaînes base64 décodées donnent `innerWidth` , `performance` , `now` et `hidden` . **6,5 s après le chargement**, toutes les **2,6 s**, si l'onglet est visible et la fenêtre **> 1024 px** : boucle `while` d'attente active de 46 ms, **allongée de 28 % à chaque passage** jusqu'à **430 ms**	Mesure en fenêtre de 1280 px : blocages de **66, 87, 115, 149, 194 puis 251 ms**, espacés de 2,6 s. Invisibles dans PageSpeed (audit court, profil mobile < 1024 px), d'où l'écart entre les mesures	Blocages croissants : la page « rame après quelques secondes », exactement la plainte du client	Script **conservé** (même déclenchement, mêmes conditions, même objet `window.__au`), mais empreinte du DOM calculée une seule fois et plus de boucle d'attente — *Supprime boucle bloquante analytics desktop*. Résultat : **0 blocage sur 30 s**

Régressions détectées par mes propres mesures, puis corrigées

#	Nature	Preuve	Correction (commit)
18	Le passage à `display=swap` (n°10) a **fait apparaître** un CLS de **0,125** en mobile : le texte s'affiche d'abord en police système, puis en Poppins, plus large, et le titre change de nombre de lignes. Avec `display=block` , le texte était invisible mais ne bougeait pas	PSI « Causes des décalages de mise en page » : `<div class="hero">` , 0,125, polices `pxiEyp8…` et `pxiByp8…`	Police de secours « Poppins Fallback » (Arial ou ses équivalents Liberation Sans / Arimo) agrandie avec `size-adjust` et recalée avec `ascent/descent/line-gap-override` . Calibrage mesuré sur les textes du site : +9,8 % en 400, +6,4 % en gras. Hauteur de la barre et du bandeau identiques à Poppins de 360 à 1280 px — *Supprime décalage dû aux polices*

#	Nature	Preuve	Correction (commit)
19	CLS résiduel de **0,034** dans la galerie : sur une colonne, le JavaScript agrandissait la carte 1 (photo 16/9) de **214 à 284 px** après le premier affichage, ce qui poussait tout le contenu suivant	PSI : `<div id="galerie">` 0,034 ; mesure des hauteurs de cartes à 412 px	`.card{aspect-ratio:4/3}` réserve la hauteur dès le premier rendu ; la fonction JavaScript et ses écouteurs sont supprimés — *Réserve hauteur des cartes en CSS*
20	Modifier l'opacité du calendrier relance la transition des 371 cases : **15 à 22 ms** de recalcul de styles par image pendant le défilement, que les variables CSS soient posées sur le conteneur ou directement sur les cases	Chronométrage des trois approches (variables CSS, styles en ligne, feuille de style) : 20, 27 et 19 ms	Effet conservé (iso-rendu), mais calcul limité aux moments où le calendrier est à moins d'un écran, et seulement si une valeur change — *Limite le calcul du fondu*. **Gain modeste** : la page est courte, donc en desktop le calendrier est presque toujours dans cette zone

Après les corrections 18 et 19, le CLS mobile vaut 0 sur 2 des 3 analyses, et **0,019** sur la troisième. Ce reste vient de l'ordre d'arrivée des polices sur le serveur de Google, et il est très en dessous du seuil « bon » de 0,1.

---

4. Optimisations majeures : problème, correction, gain mesuré

Optimisation	Indicateur visé	Avant	Après	Gain mesuré
Images (n°1, 3, 4, 5)	Poids, LCP	16 376 Kio ; LCP mobile 67,1 s, desktop 10,9 s	559 Kio (mobile) / 210 Kio (desktop) ; LCP 3,0 s / 0,6 s	**−96,6 % / −98,7 %** de poids ; LCP **÷ 22** en mobile, **÷ 18** en desktop
Chaîne critique CSS et polices (n°6 à 11)	FCP, requêtes	FCP 2,4 s (mobile), 0,9 s (desktop) ; 19 requêtes	FCP 2,4 s (mobile), 0,6 s (desktop) ; 9 requêtes	Desktop −0,3 s ; **mobile : aucun gain sur le FCP** (voir ci-dessous) ; −10 requêtes
Suppression de UIKit (n°12, 13, 16)	TBT	100 ms (mobile), 320 ms (desktop) ; blocages de 691 et 454 ms au chargement	0 ms ; aucun blocage	**TBT −100 %**
Reflows au scroll (n°14, 20)	Fluidité du défilement	19,2 ms de JavaScript par événement scroll	0,03 ms par événement, plus un calcul de 0,12 ms par image	**JavaScript ÷ 640**
`analytics.js` (n°17)	Blocages après 6,5 s en desktop	66 → 251 ms toutes les 2,6 s ; 2 007 ms bloqués sur 30 s	0 blocage	**−100 %**
Polices et cartes (n°18, 19)	CLS	0,125 puis 0,034 (régressions introduites par mes changements)	0 (médiane)	Régressions supprimées

Ce qui n'a rien changé : le **FCP mobile** reste à 2,4 s. En connexion 4G lente simulée, le premier affichage attend toujours deux feuilles de style bloquantes : `styles.css` et la feuille Google Fonts, sur un domaine tiers qui demande sa propre connexion. Pour le faire baisser, il faudrait héberger la police sur le site ou intégrer le CSS critique dans le HTML (voir section 6).

---

5. Contrôle de l'iso-rendu

Captures avant/après : `audit/iso-avant-desktop.png`, `audit/iso-apres-desktop.png` (et la même chose en mobile). Points vérifiés :

- Même mise en page, mêmes couleurs, mêmes ombres. La texture du bandeau, l'animation d'arrivée des cartes et le fondu du calendrier sont conservés. Pour le calendrier, les opacités sont identiques à l'original à 0,0004 près, ce qui est invisible.

- Filtre par mot-clé, liste de 320 suggestions, calendrier de 371 jours, fenêtre des consignes et suivi d'audience (`window.__au`) fonctionnent tous.
- Les cartes gardent toutes la hauteur de la plus grande, comme avant, mais c'est maintenant le CSS qui la fixe au lieu du JavaScript.
- Seule différence volontaire : l'image des pages de détail ne déborde plus de l'écran (c'était un défaut, cf. guide A1).
- Le flou `backdrop-filter` de la barre de navigation est coûteux à afficher, mais il fait partie du rendu : il est conservé.

---

6. Optimisations impossibles sur GitHub Pages

Ces optimisations relèvent du serveur, que je ne contrôle pas sur un hébergement statique :

- **Cache-Control** : `max-age=31536000, immutable` sur les images, le CSS et le JS, et une revalidation courte sur le HTML. GitHub Pages impose `max-age=600` : PSI signale encore « Utiliser des durées de mise en cache efficaces, 492 Kio ».
- **Empreinte dans les noms de fichiers** (`styles.a3f9c1.css`) pour pouvoir mettre un cache long sans risquer de servir une version périmée. Cela demande un outil de build, interdit ici.
- **Brotli** au lieu de gzip pour les fichiers texte (environ 15 à 20 % de mieux).
- **Polices hébergées sur le site** : cela supprimerait la connexion à deux domaines tiers (googleapis et gstatic). C'est la piste la plus prometteuse pour le FCP mobile, qui n'a pas bougé.
- **WebP / AVIF** avec `<picture>` : environ −30 % (WebP) à −50 % (AVIF) supplémentaires sur les photos. PSI estime encore 191 Kio d'économies possibles sur les images. C'est faisable hors ligne avec Squoosh.
- **CDN** pour rapprocher les fichiers des visiteurs, et **HTTP/2 ou HTTP/3** (déjà fournis par GitHub Pages).

---

7. Conclusion

Le site passe de **61 à 89** en mobile et de **52 à 100** en desktop (médianes de 3 analyses PageSpeed Insights), sans changement visible pour le visiteur.

- **Le poids** est le gain le plus spectaculaire : 16 Mo d'images surdimensionnées, en double ou invisibles, ramenés à 0,2 – 0,6 Mo. C'est ce qui fait chuter le LCP de 67 s à 3 s en mobile.
- **Le JavaScript** était le vrai responsable de la sensation de lenteur. UIKit bloquait le chargement (TBT 320 ms en desktop), le gestionnaire de défilement dépassait le budget d'une image, et `analytics.js` bloquait la page de plus en plus longtemps. Ce dernier problème était invisible pour PageSpeed : il ne se déclenche qu'après 6,5 s, et seulement sur un écran de plus de 1024 px. C'est un enregistrement long en fenêtre large qui l'a révélé, et il correspond exactement à la plainte du client (« ça rame après quelques secondes »).
- **Mesurer après chaque changement** m'a permis de repérer deux régressions que j'avais moi-même introduites : le CLS de 0,125 causé par `display=swap`, puis celui de 0,034 causé par l'égalisation des cartes en JavaScript. Je les ai corrigées en traitant la cause.
- **Limites :** le FCP mobile n'a pas bougé (2,4 s), et le fondu du calendrier reste coûteux par nature pendant qu'on le fait défiler. Il faudrait héberger la police sur le site et pouvoir régler les en-têtes de cache, ce que GitHub Pages ne permet pas.